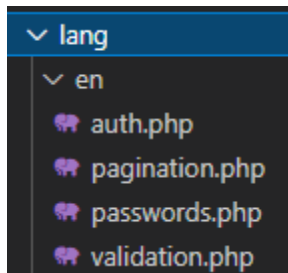


LARAVEL : Les fichiers de langage

On a deux façons de gérer les langues.

Clés et valeurs

On a déjà parlé des fichiers de langage. Lorsque Laravel est installé on a cette architecture :

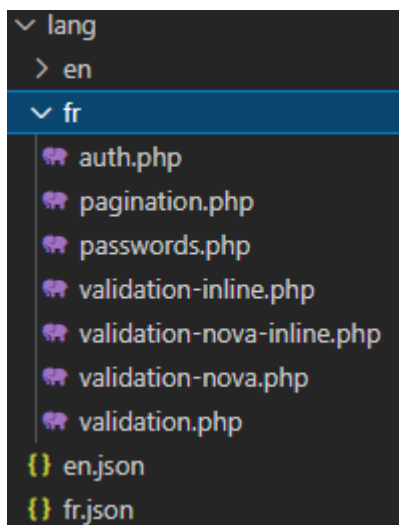


Il n'est prévu au départ que la langue anglaise (**en**) avec 4 fichiers. Pour ajouter une langue, il suffit de créer un nouveau dossier, par exemple **fr** pour le Français.

Il faut évidemment avoir le même contenu comme nous allons le voir.

Vous n'allez pas être obligé de faire toute cette traduction vous-même ! Certains s'en sont déjà chargés avec [ce package](#).

On va ajouter la traduction française à partir de ce dépôt :



Attention au fichier fr.json qui doit être extrait du dossier pour être placé à la racine.

Les fichiers présents sont constitués exactement de la même manière mis à part trois fichiers qui servent pour des installations supplémentaires comme Nova). Prenons le plus léger : **pagination.php**, pour l'anglais on a :

```
return [  
'previous' => '&laquo; Previous',  
'next' => 'Next &raquo;',  
];
```

On voit qu'on se contente de renvoyer un tableau avec des clés et des valeurs. On ne peut pas faire plus simple ! Si on prend le même fichier en version française :

```
return [  
'previous' => '&laquo; Précédent',  
'next' => 'Suivant &raquo;',  
];
```

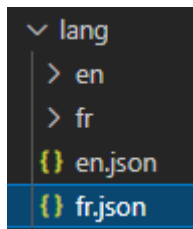
On retrouve bien sûr les mêmes clés avec des valeurs adaptées au langage.

Les textes

Le système clé/valeur peut devenir rapidement lourd et confus lorsqu'on a beaucoup de textes à traduire. Laravel propose une autre approche qui consiste à utiliser la traduction par défaut comme clé.

Si la langue par défaut est l'anglais par exemple, on aura le texte anglais directement dans le code. Les autres traductions se trouveront dans des fichiers JSON.

Par exemple pour le français, on a le fichier **fr.json** :



Ce fichier contient des éléments de ce type :

```
"90 Days": "90 jours",
```

On a :

- **90 Days** : le texte en langage par défaut présent dans le code
- **90 jours** : la traduction en français

Les deux systèmes peuvent cohabiter et se compléter.

Configuration de la locale

La configuration de la locale est effectuée dans le fichier **config/app.php** :

```
'locale' => 'en',
```

On peut changer cette locale en cours d'exécution si nécessaire :

```
App::setLocale('fr');
```

Si une traduction n'est pas trouvée dans la locale actuelle alors on va chercher la traduction de la locale par défaut définie également dans le fichier **config/app.php** :

```
'fallback_locale' => 'en',
```

On peut connaître la locale actuelle :

```
$locale = App::getLocale();
```

Et même tester si on a une certaine locale :

```
if (App::isLocale('fr')) {  
    //  
}
```

Dans le code

L'helper __

Concrètement au niveau du code, on utilise l'helper `__`. Il suffit de préciser la clé comme paramètre (la clé pour le système clé-valeur et le texte dans la langue d'origine pour le second système). Prenons un exemple :

```
$info = __('Posts for category: ');
```

Si la locale est **en** c'est ce texte qui sera directement affiché. Si la locale est **fr** c'est le texte correspondant qu'on trouve dans le fichier **fr.json** qui sera affiché :

```
"Posts for category: ": "Articles pour la catégorie : ",
```

Blade

On peut utiliser l'helper `__` dans une vue Blade :

```
{{ __('Get In Touch With Us') }}
```

Mais il est plus pratique d'utiliser la directive **@lang** :

```
@lang('Get In Touch With Us')
```

Le pluriel

La gestion du pluriel n'est pas toujours facile. On peut prévoir le texte du singulier et celui du pluriel séparés par le caractère | :

```
'pommes' => 'une pomme|plusieurs pommes',
```

On peut affiner :

```
'pommes' => '{0} aucune pomme|[1,10] quelques pommes|[10,*] beaucoup de pommes',
```

On utilise dans le code la fonction **trans_choice**. Voici un exemple tiré d'une application :

```
{{ trans_choice(__('comment|comments'), $post->valid_comments_count) }}
```

Selon la valeur de **\$post->valid_comments_count**, autrement dit le nombre de commentaires valides, on prend le singulier ou le pluriel grâce à **trans_choice** et dans la langue voulue grâce à l'helper `__`.